CARBON COMMUTE CALCULATOR

Presented By: Bhargavi Kale and Isabella Velazquez-Abelardo

COMP 705/805.M1 Spring 2026

SUSTAINABILITY RELEVANCE

Our Goal

To help users recognize exactly how much their individual commute contributes to environmental changes.

- Transportation = major CO_2 contributor
- Daily commuting decisions matter
- Most people lack emission awareness

Google Maps Routes API

- Calculates real driving distance
- Enables route visualization

EPA Vehicle Dataset (CSV)

- Government vehicle data source
- Provides fuel type & vehicle information
- Ensures realistic carbon emission estimates

USE CASES

1. Vehicle Management
   - Select Make → Model → Year (each step enabled based on previous selection)
   - Fetch MPG & fuel type and save the vehicle details to VehicleModel

2. Route Visualization
   - Display origin → destination route on map UI (Google Maps)

3. Carbon Emission Calculation
   - Compute CO_2 emission using distance + MPG + fuel type
   - Display emission summary and save to Commute Model

4. Dashboard
   - Each trip is saved with vehicle, fuel type, origin, destination, and distance
   - UI displays history with calculated CO_2 emissions for every commute

DATABASE MODEL TO USE CASE MAPPING

USE CASE 1: VEHICLE PROFILE MANAGEMENT

Select/Save unique vehicles from dynamic EPA dropdowns

USE CASES 3 & 4: EMISSION CALCULATION & DASHBOARD

Run trip scenarios (calc CO2) and review historical statistics

VehicleModel
(cite: `models.py)

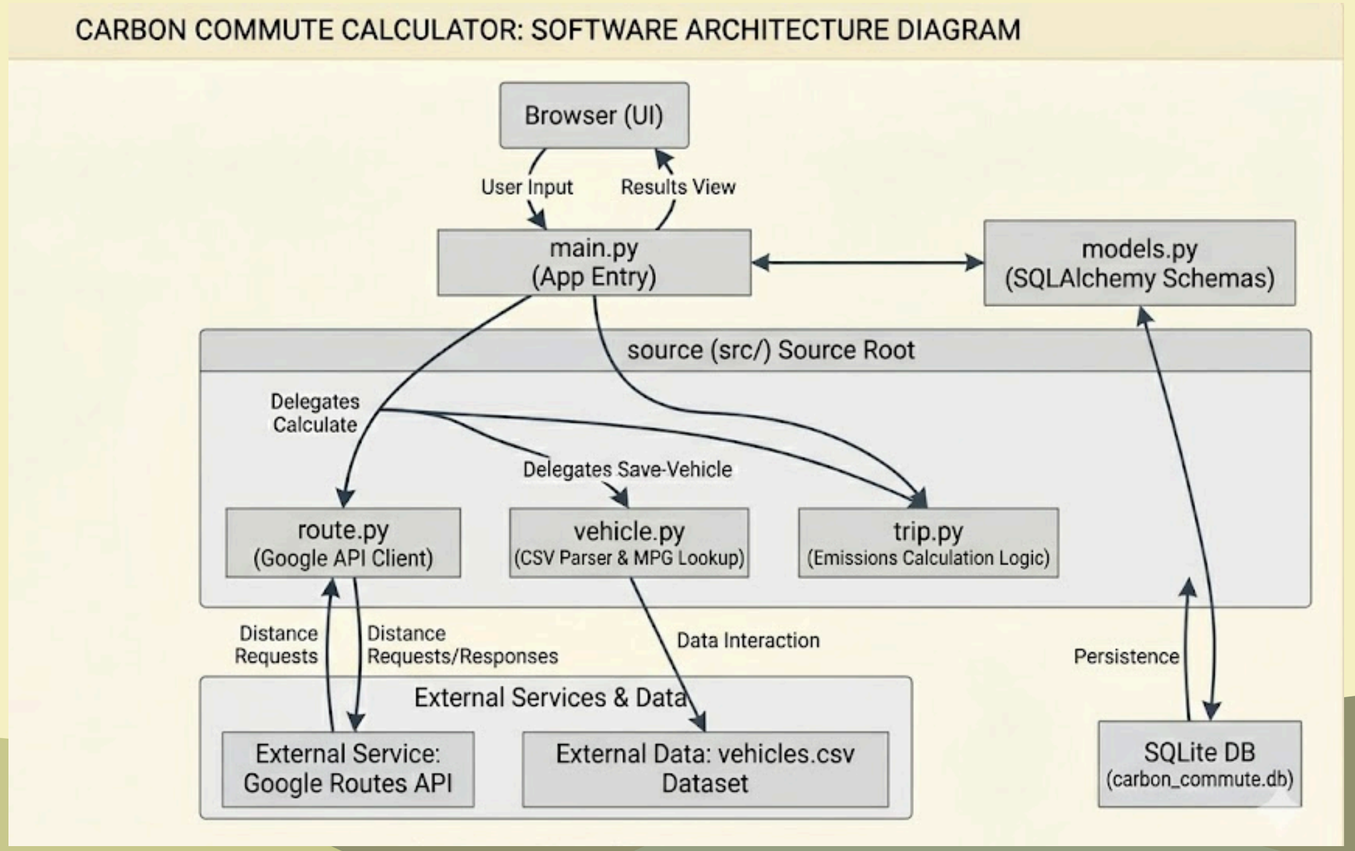
id	**PK**
make	
model	
year	
economy_id	
fuel_type	
combined_mpg	

Commute
(cite: `models.py)

id	**PK**
vehicle_name	
fuel_type	
origin	
destination	
distance_miles	
distance_km	
total_emissions	
emissions_per_person	

ARCHITECTURE

```
carbon-commute/
│
├── data/
│   └── vehicles.csv
├── docs/
│   └── sprints.md
│   └── Projectoutline.md
├── instance/
│   └── carbon_commute.db
├── static/
│   └── style.css
├── templates/
│   ├── home.html
│   └── index.html
│   └── vehicle.html
├── venv
├── .env
├── .gitignore
├── dev-requirements.txt
├── main.py
├── models.py
├── README.md
├── requirements.txt
```



TECHNOLOGY STACK

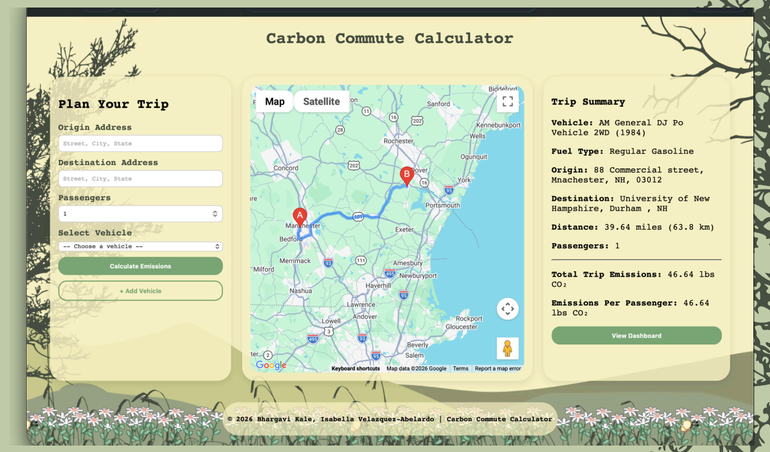
```
# Linting + formatting +
ruff==0.9.6
# Deep static analysis w
pylint==3.3.4
# Static type checking
mypy==1.14.1
#pytest
pytest
```

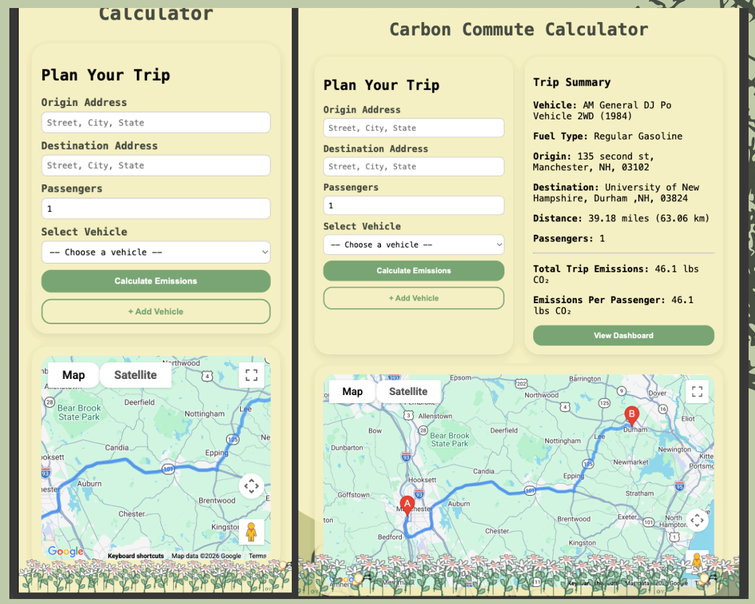
```
Flask==3.1.0
Flask-SQLAlchemy==3.1.1
requests==2.32.3
python-dotenv==1.0.1
pandas==2.0.3
numpy<2        You, 3 week
```

- Backend: Python 3.12, Flask Framework, SQLAlchemy ORM.
- Frontend: HTML5, CSS3 , Jinja2 Templates, Javascript.
- External APIs: Google Maps & Routes API.
- Data Processing: Pandas for vehicle dataset management.

RESPONSIVE MULTI-LAYOUT SYSTEM (BHARGAVI)

- Built responsive grid for trip form, map, and summary
- Laptop: 3-column full dashboard view
- Tablet: 2-column layout with map repositioned
- Mobile: Vertical stacked layout for readability
- Handled layout issues using CSS media queries





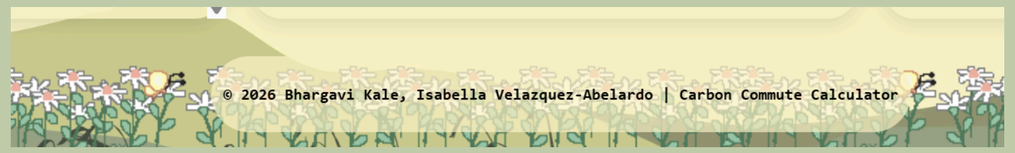
ADAPTIVE UI DESIGN SYSTEM- ISABELLA

CSS VARIABLES

```
--primary      #464f41
--accent       #7aa874
--accent-dark  #5e8c5a
--panel-bg     rgba(249,242,196,…)
--panel-radius 25px
--max-width    1500px
```

UI Components

- Panels (cream, rounded 25px)
- Buttons (green, 15px radius)
- Map (Google Maps Embeded)
- Flower GIF Footer



DEMO

CONTRIBUTIONS- BELLA

Frontend Design & User Experience:

- Designed initial application layout
- Experimented with color themes & visual styling
- Created vehicle selection background design

Interface Development

- Created HTML templates connected with Jinja
- Integrated database data into UI
- Implemented responsive Google Maps display
- Adjusted layouts for usability and consistency

CONTRIBUTIONS – BHARGAVI

- Set up project environment (venv, SQLAlchemy, code structure) and worked on dataset loading using pandas; stored vehicle data in database
- Developed vehicle screen with cascading dropdowns (make, model, year) to reduce input errors
- Integrated trip logic to calculate distance and MPG into carbon emission calculation
- Implemented responsive grid system using CSS Grid and media queries for all screen sizes
- Added back navigation in vehicle and dashboard screens for better user flow
- Handled edge cases using flash messages (auto-dismiss after 4 seconds)
- Wrote test cases for core functions and API endpoints
- Fixed code quality issues, created documentation using pdoc, and cleaned unused code.